

Git Hands-On Lab Assignments Record & Solution Guide

This document contains the full execution instructions, exact terminal commands, and output verification logs for Git Assignments 1 through 5. Follow these steps sequentially in Git Bash to complete the labs and maintain a complete lab record.

Note on Credentials: Create a free account on GitHub or GitLab. Do not use organizational or company credentials.

Assignment 1: Git Setup, Editor Integration & Basic Workflow

Objectives: Familiarize with Git setup, configure Notepad++ as default editor, track files, and interact with remote repositories.

Step 1: Setup Machine with Git Configuration

1. Check if Git client is installed properly:

```
git --version
```

Expected Output: git version 2.42.0.windows.1 (or current installed version)

2. Configure user level configuration (Name and Email ID):

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

3. Verify user global configuration:

```
git config --list
```

Step 2: Integrate Notepad++ to Git as Default Editor

1. Check if Notepad++ executes from Git Bash. If not, add Notepad++ directory (e.g., C:\Program Files\Notepad++) to system Environment Variables (Path) via Control Panel > System > Advanced System Settings.

2 & 3. Create alias for Notepad++ in bash profile:

```
echo 'alias npp="/c/Program Files/Notepad++/notepad++.exe" -multiInst  
-notabbar -nosession"' >> ~/.bashrc  
source ~/.bashrc
```

4. Configure Notepad++ as core Git editor:

```
git config --global core.editor "'C:/Program  
Files/Notepad++/notepad++.exe' -multiInst -notabbar -nosession"
```

5. Verify default editor configuration:

```
git config --global -e
```

Step 3: Add File to Source Code Repository

1 & 2. Create and initialize "GitDemo" repository:

```
mkdir GitDemo  
cd GitDemo  
git init  
ls -la
```

3, 4 & 5. Create "welcome.txt" and check content:

```
echo "Welcome to Git Hands-on Lab" > welcome.txt  
ls  
cat welcome.txt
```

6 & 7. Check status and stage file for tracking:

```
git status  
git add welcome.txt
```

8 & 9. Commit changes with default editor (Notepad++) and check status:

```
git commit
```

Notepad++ opens. Type multi-line commit message ("Initial commit: Added welcome.txt"), save, and close Notepad++.

```
git status
```

10, 11 & 12. Connect and sync with remote repository on GitLab/GitHub:

```
git remote add origin https://gitlab.com/your-username/GitDemo.git
```

```
git pull origin master --allow-unrelated-histories
git push -u origin master
```

Assignment 2: Ignoring Unwanted Files using .gitignore

Objectives: Ignore unwanted files and directories using .gitignore.

Step-by-Step Execution

1. Create sample log files and a log directory:

```
touch app.log error.log debug.log
mkdir log
touch log/system.log
```

2. Check status to observe untracked files:

```
git status
```

3. Create and configure .gitignore file:

```
echo "*.log" > .gitignore
echo "log/" >> .gitignore
cat .gitignore
```

4. Verify if log files and folders are ignored:

```
git status
```

Expected Output: Only .gitignore is listed as untracked. All .log files and log/ folder are ignored!

5. Commit and push .gitignore:

```
git add .gitignore
git commit -m "Add .gitignore rules for log files and directory"
git push origin master
```

Assignment 3: Branching, Merging & P4Merge Visual Tool

Objectives: Branching workflow, visual diff inspection using P4Merge, merging, and branch

deletion.

Branching

1 & 2. Create "GitNewBranch" and list all local/remote branches:

```
git branch GitNewBranch  
git branch -a
```

3, 4 & 5. Switch to branch, add file content, commit, and verify status:

```
git checkout GitNewBranch  
echo "Feature updates in branch" > feature.txt  
git add feature.txt  
git commit -m "Add feature.txt in GitNewBranch"  
git status
```

Merging

1 & 2. Switch to master and inspect CLI differences:

```
git checkout master  
git diff master..GitNewBranch
```

3. Inspect visual differences using P4Merge tool:

```
git config --global diff.tool p4merge  
git config --global difftool.p4merge.path "C:/Program  
Files/Perforce/p4merge.exe"  
git difftool master..GitNewBranch
```

4 & 5. Merge branch to master and observe graph log:

```
git merge GitNewBranch  
git log --oneline --graph --decorate
```

6. Delete merged branch and verify status:

```
git branch -d GitNewBranch  
git status
```

Assignment 4: Merge Conflict Resolution with 3-Way Merge

Objectives: Implement conflict resolution using 3-way merge tool when master and branch have conflicting changes.

Step-by-Step Execution

1, 2, 3 & 4. Verify clean master, create branch "GitWork", and commit "hello.xml":

```
git status
git checkout -b GitWork
echo "<config><version>1.0-branch</version></config>" > hello.xml
git add hello.xml
git commit -m "Add hello.xml on GitWork branch"
```

5, 6 & 7. Switch to master, create conflicting "hello.xml", and commit:

```
git checkout master
echo "<config><version>1.0-master</version></config>" > hello.xml
git add hello.xml
git commit -m "Add hello.xml on master branch with conflicting text"
```

8, 9 & 10. Inspect log graph and visual differences:

```
git log --oneline --graph --decorate --all
git difftool master..GitWork
```

11 & 12. Trigger merge to generate conflict and observe markup:

```
git merge GitWork
```

13. Launch 3-Way merge tool (P4Merge) to resolve conflict:

```
git config --global merge.tool p4merge
git config --global mergetool.p4merge.path "C:/Program
Files/Perforce/p4merge.exe"
git mergetool
```

Select the desired resolved lines in P4Merge GUI, save, and exit.

14, 15 & 16. Commit resolution and ignore P4Merge .orig backup files:

```
git status
git commit -m "Resolved merge conflict in hello.xml"
echo "*.orig" >> .gitignore
git add .gitignore
git commit -m "Ignore P4Merge .orig backup files"
```

17, 18 & 19. Clean up merged branch and view final history log:

```
git branch -a  
git branch -d GitWork  
git log --oneline --graph --decorate
```

Assignment 5: Repository Clean-Up and Remote Synchronization

Objectives: Execute final clean-up and push all changes back to remote Git repository.

Step-by-Step Execution

1 & 2. Verify clean state and list available branches:

```
git status  
git branch -a
```

3. Pull remote Git repository to master:

```
git pull origin master
```

4 & 5. Push pending changes to remote repository and verify:

```
git push origin master
```

Expected Output: Everything up-to-date / All commits successfully pushed to origin/master on GitLab/GitHub.